Practical No 01

Aim : Install and understand Docker container, Node.js, Java and perform necessary software installation on local machine/create instance on Cloud to run blockchain application

Setting up Docker containers, Node.js, Java, Hyperledger Fabric, and Ethereum on a local machine or a cloud instance involves a series of steps. Below is a comprehensive guide that walks you through setting up these tools on a local machine or a cloud instance such as AWS EC2 or Google Cloud VM.

Step 1: Install Docker

Docker is essential for running containerized services like Hyperledger Fabric and Ethereum.

On Local Machine:

1. Install Docker:
   - Windows:
     - Download Docker Desktop from Docker's website.
     - Run the installer and follow the on-screen instructions.
   - Verify Docker Installation: After installation, you can check the Docker version:

```
docker --version
```

2. Start Docker (if it isn't already running):
   - On Windows/macOS, Docker Desktop should start automatically.
   - Test Docker: Run a test container to check if Docker is working properly:

```
docker run hello-world
```

Step 2: Install Node.js

Node.js is required for interacting with both Hyperledger Fabric and Ethereum from JavaScript/TypeScript-based applications.

On Local Machine:

1. Install Node.js:
   - Windows/macOS: Visit Node.js official site and download the installer for your OS.
   - Verify Node.js Installation:

```
node --version
npm --version
```

Step 3: Install Java (For Hyperledger Fabric)

Java is needed for interacting with Hyperledger Fabric SDKs.

On Local Machine:

1. Install Java:
   - Windows/macOS: Download Java from AdoptOpenJDK or Oracle's official website.
   - Verify Java Installation:

```
java -version
```

Practical No 02

Aim : Create and deploy a block chain network using Hyperledger Fabric SDK for Java
Code :

```
import hashlib, time, json

class Block:
    def __init__(self, i, d, p):
        self.i = i
        self.t = time.time()
        self.d = d
        self.p = p
        self.h = hashlib.sha256(f"{i}{self.t}{json.dumps(d)}{p}".encode()).hexdigest()

class Blockchain:
    def __init__(self):
        self.c = [Block(0, "Genesis", "0")]

    def add(self, d):
        self.c.append(Block(len(self.c), d, self.c[-1].h))

    def valid(self):
        for i in range(1, len(self.c)):
            if self.c[i].p != self.c[i-1].h:
                return False
        return True

bc = Blockchain()
bc.add({"Asset":"A1","Owner":"Tom"})
bc.add({"Asset":"A2","Owner":"Alice"})

for b in bc.c:
    print(b.i, b.d, b.h)

print("Valid:", bc.valid())
```

Output :

```
0 Genesis
be24c5c4c50c19dbe232101cfa430f2ac2dc53fe8c455c5526950baa4c5c1e5a
1 {'Asset': 'A1', 'Owner': 'Tom'}
7c24cb34d063449beb5a6250c85048ff57cac6400d4330602c09a8f641a84f3a
2 {'Asset': 'A2', 'Owner': 'Alice'}
939958b19ba61e9f53b326c1116e135a3895ac0a4872521a8c1894309c965faa
Valid: True
```

Practical No 03

Aim : Interact with a blockchain network. Execute transactions and requests against a blockchain network by creating an app to test the network and its rules

Code :

```
import hashlib, time

class Block:
    def __init__(s,i,d,p):
        s.i,s.t,s.d,s.p=i,time.time(),d,p
        s.h=hashlib.sha256(f"{i}{s.t}{d}{p}".encode()).hexdigest()

class Blockchain:
    def __init__(s):
        s.c=[Block(0,"Genesis","0")]
    def transact(s,d):
        s.c.append(Block(len(s.c),d,s.c[-1].h))
    def query(s,i):
        return s.c[i].d if i<len(s.c) else None
    def valid(s):
        return all(s.c[i].p==s.c[i-1].h for i in range(1,len(s.c)))

bc=Blockchain()
bc.transact({"user":"A","amount":100})
bc.transact({"user":"B","amount":50})

print(bc.query(1))
print(bc.query(2))
print(bc.valid())
```

Output :

```
{'user': 'A', 'amount': 100}
{'user': 'B', 'amount': 50}
True
```

Practical No 04

Aim : Deploy an asset-transfer app using block chain. Learn app development within a Hyperledger Fabric network

Code :

```
import hashlib,time

class B:
    def __init__(s,i,d,p): s.i,s.d,s.p=i,d,p;
s.h=hashlib.sha256(f"{i}{time.time()}{d}{p}".encode()).hexdigest()

class BC:
    def __init__(s): s.c=[B(0,"Genesis","0")]
    def add(s,d): s.c.append(B(len(s.c),d,s.c[-1].h))

class App:
    def __init__(s): s.bc,s.a=BC(),{}
    def create(s,i,o,v): s.a[i]={"owner":o,"value":v}; s.bc.add(("create",i,o,v))
    def transfer(s,i,n): s.a[i]["owner"]=n; s.bc.add(("transfer",i,n))

app=App()
app.create("A1","Alice",500)
app.transfer("A1","Bob")
print(app.a["A1"])
print(app.bc.c[1].d, app.bc.c[2].d)
```

Output :

```
{'owner': 'Bob', 'value': 500}
('create', 'A1', 'Alice', 500)
('transfer', 'A1', 'Bob')
```

Practical No 05

Aim : Use blockchain technology to track fitness club rewards points.

Code :

```
import hashlib, time

class Block:
    def __init__(s,i,d,p):
        s.i,s.t,s.d,s.p=i,time.time(),d,p
        s.h=hashlib.sha256(f"{i}{s.t}{d}{p}".encode()).hexdigest()

class Chain:
    def __init__(s): s.c=[Block(0,"Genesis","0")]
    def add(s,d): s.c.append(Block(len(s.c),d,s.c[-1].h))

class Rewards:
    def __init__(s): s.c,s.r=Chain(),{}
    def reward(s,m,p):
        s.r[m]=s.r.get(m,0)+p
        s.c.add((m,p,s.r[m]))

if __name__=="__main__":
    r=Rewards()
    r.reward("m1",10); r.reward("m2",20); r.reward("m1",15)
    print(r.r)
    for b in r.c.c:
        print(b.i,b.d,b.h)
```

Output :

```
{'m1': 25, 'm2': 20}
0 Genesis 21f985ad031cada7c01b10285e1747cd0690a083ecfaba39cedc58d4b6b6a1b0
1 ('m1', 10, 10) 97e6d42f2b76a3c89bba2a86340859482f3d3f49ba337762381239c6ded9f533
2 ('m2', 20, 20) 6041683460f6ccd632d989d7d1fe5439eb0c88718867a55d069efe275ae600b0
3 ('m1', 15, 25) 9fd1b8497eda07834c4726ebe3e8a73988de1c0e7869fadcb23389201e10c92b
```

Practical No 06

Aim: Build a web app that uses Hyperledger Fabric to track and trace member rewards

Code :

```
import hashlib, time

class Block:
    def __init__(s,i,d,p):
        s.h=hashlib.sha256(f"{i}{time.time()}{d}{p}".encode()).hexdigest()
        s.d=d

class RewardChain:
    def __init__(s):
        s.c=[Block(0,"Genesis","0")]
        s.r={}
    def reward(s,m,p):
        s.r[m]=s.r.get(m,0)+p
        s.c.append(Block(len(s.c),{"member":m,"points":p,"total":s.r[m]},s.c[-1].h))
    def trace(s,m):
        return [b.d for b in s.c if isinstance(b.d,dict) and b.d["member"]==m]

rc=RewardChain()
rc.reward("M1",10)
rc.reward("M2",20)
rc.reward("M1",15)

print("Member M1 trace:", rc.trace("M1"))
print("Member M2 trace:", rc.trace("M2"))
```

Output :

```
Member M1 trace: [{'member': 'M1', 'points': 10, 'total': 10},
{'member': 'M1', 'points': 15, 'total': 25}]
Member M2 trace: [{'member': 'M2', 'points': 20, 'total': 20}]
```

Practical No 07

Aim: Develop a Car auction network: A Hello World example with Hyperledger Fabric Node SDK and IBM Blockchain Starter Plan. Use Hyperledger Fabric to invoke chaincode while storing results and data in the starter plan

Code :

```
import hashlib,time
class Block:
    def __init__(s,i,d,p): s.i,s.t,s.d,s.p=i,time.time(),d,p;
s.h=hashlib.sha256(f"{i}{s.t}{d}{p}".encode()).hexdigest()
class Blockchain:
    def __init__(s): s.c=[Block(0,"Genesis","0")]
    def invoke(s,d): s.c.append(Block(len(s.c),d,s.c[-1].h))
    def query(s,k): return [b.d for b in s.c if isinstance(b.d,dict) and b.d.get("car_id")==k]
class Auction:
    def __init__(s): s.bc=Blockchain(); s.cars={}
    def add_car(s,id,owner,price): s.cars[id]={"owner":owner,"price":price};
s.bc.invoke({"car_id":id,"owner":owner,"price":price,"action":"add"})
    def bid(s,id,bidder,bid):
        car = s.cars.get(id)
        if car and bid>car["price"]:
            car["owner"]=bidder
            car["price"]=bid
            s.bc.invoke({"car_id":id,"owner":bidder,"price":bid,"action":"bid"})
    def history(s,id): return s.bc.query(id)
app=Auction()
app.add_car("C1","Alice",5000)
app.bid("C1","Bob",5500)
app.bid("C1","Charlie",5400)
print(app.cars["C1"])
print(app.history("C1"))
```

Output :

```
{'owner': 'Bob', 'price': 5500}
[{'car_id': 'C1', 'owner': 'Alice', 'price': 5000, 'action': 'add'},
{'car_id': 'C1', 'owner': 'Bob', 'price': 5500, 'action': 'bid'}]
```

Practical No 08

Aim : Develop an IoT asset tracking app using Blockchain. Use an IoT asset tracking device to improve a supply chain by using Block chain, IoT devices, and Node-RED

Code :

```
import hashlib, time, random

class Block:
    def __init__(s,i,d,p):
        s.i,s.t,s.d,s.p=i,time.time(),d,p
        s.h=hashlib.sha256(f"{i}{s.t}{d}{p}".encode()).hexdigest()

class Blockchain:
    def __init__(s): s.c=[Block(0,"Genesis","0")]
    def add(s,d): s.c.append(Block(len(s.c),d,s.c[-1].h))
    def query(s,asset_id): return [b.d for b in s.c if isinstance(b.d,dict) and
b.d.get("asset_id")==asset_id]

class IoTAssetApp:
    def __init__(s): s.bc=Blockchain(); s.assets={}
    def register_asset(s,asset_id,location,status):
        s.assets[asset_id]={"location":location,"status":status}
        s.bc.add({"asset_id":asset_id,"location":location,"status":status,"action":"register"})
    def update_asset(s,asset_id,location,status):
        if asset_id in s.assets:
            s.assets[asset_id]={"location":location,"status":status}
            s.bc.add({"asset_id":asset_id,"location":location,"status":status,"action":"update"})

app = IoTAssetApp()

app.register_asset("A1","Warehouse", "Idle")
app.update_asset("A1","In Transit", "Moving")
app.update_asset("A1","Store","Delivered")
app.register_asset("A2","Factory","Idle")
app.update_asset("A2","Warehouse","Moving")

print("A1 History:", app.bc.query("A1"))
print("A2 History:", app.bc.query("A2"))
```

Output :

```
A1 History: [{'asset_id': 'A1', 'location': 'Warehouse', 'status':
'Idle', 'action': 'register'}, {'asset_id': 'A1', 'location': 'In
Transit', 'status': 'Moving', 'action': 'update'}, {'asset_id': 'A1',
'location': 'Store', 'status': 'Delivered', 'action': 'update'}]
A2 History: [{'asset_id': 'A2', 'location': 'Factory', 'status': 'Idle',
'action': 'register'}, {'asset_id': 'A2', 'location': 'Warehouse',
'status': 'Moving', 'action': 'update'}]
```

Practical No 09

Aim: Create a global finance block chain application with IBM Block chain Platform Extension for VS Code. Develop a Node.js smart contract and web app for a Global Finance with block chain use case

Code :

```
import hashlib, time

class Block:
    def __init__(self, i, t, d, p):
        self.i, self.t, self.d, self.p = i, t, d, p
        self.h = hashlib.sha256(f"{i}{t}{d}{p}".encode()).hexdigest()

class Blockchain:
    def __init__(self):
        self.c = [Block(0, time.time(), "Genesis", "0")]

    def add(self, d):
        b = self.c[-1]
        self.c.append(Block(len(self.c), time.time(), d, b.h))

    def show(self):
        for b in self.c:
            print(b.i, b.d, b.h, b.p, sep="\n")
            print("-"*30)

bc = Blockchain()
bc.add("USA sends $2000 to INDIA")
bc.add("INDIA sends $500 to UK")
bc.add("UK sends $1000 to USA")
bc.show()
```

Output :

```
0
Genesis
85dd948f719508e72553528625acb8acefd6c535998083927b115ca0fada01b2
0
------------------------------
1
USA sends $2000 to INDIA
c3f1329f8dd428c3d4bba7f8c3b2313734f5c8cab345df8dfd5638fa89894b94
85dd948f719508e72553528625acb8acefd6c535998083927b115ca0fada01b2
------------------------------
```

```
2
INDIA sends $500 to UK
53b3a792efbc59893e0f3db2ab99b2d6a37e997fe17fe5d77eb17bcdfb257d2c
c3f1329f8dd428c3d4bba7f8c3b2313734f5c8cab345df8dfd5638fa89894b94
------------------------------
3
UK sends $1000 to USA
f65fb833402297034d35e0c261ac98dd8d9f3c297e7dba8eaf972c70884ff78d
53b3a792efbc59893e0f3db2ab99b2d6a37e997fe17fe5d77eb17bcdfb257d2c
------------------------------
```

Practical No 10

Aim: Develop a voting application using Hyperledger and Ethereum

Code :

```
import hashlib, time

class Block:
    def __init__(s,i,d,p):
        s.i,s.t,s.d,s.p=i,time.time(),d,p
        s.h=hashlib.sha256(f"{i}{s.t}{d}{p}".encode()).hexdigest()

class Blockchain:
    def __init__(s): s.c=[Block(0,"Genesis","0")]
    def add(s,d): s.c.append(Block(len(s.c),d,s.c[-1].h))
    def history(s): return [b.d for b in s.c if isinstance(b.d,dict)]

class VotingApp:
    def __init__(s): s.bc=Blockchain(); s.votes={}
    def vote(s,voter,candidate):
        if voter in s.votes:
            return "Voter has already voted"
        s.votes[voter]=candidate
        s.bc.add({"voter":voter,"candidate":candidate})
        return "Vote cast"
    def tally(s):
        t={}
        for c in s.votes.values(): t[c]=t.get(c,0)+1
        return t

app = VotingApp()

print(app.vote("Alice","Candidate1"))
print(app.vote("Bob","Candidate2"))
print(app.vote("Charlie","Candidate1"))
print(app.vote("Alice","Candidate2"))
print("\nVote Tally:", app.tally())
print("\nBlockchain History:", app.bc.history())
```

Output :

```
Vote cast
Vote cast
Vote cast
Voter has already voted

Vote Tally: {'Candidate1': 2, 'Candidate2': 1}

Blockchain History: [{'voter': 'Alice', 'candidate': 'Candidate1'},
{'voter': 'Bob', 'candidate': 'Candidate2'}, {'voter': 'Charlie',
'candidate': 'Candidate1'}]
```